Ricart et Agrawala's algorithme for mutual exclusion on a clique *

V.Marangozova-Martin

1 Problem

There are several processes, represented by the vertices of a clique. Upon request (of a client), they will ask the access to a critical section. To decide whether they have the right to go or not, they will interact with the other processes in the system. A process may issue multiple requests to the critical section, sequentially. If there are two conflicting requests, they will be compared by their timestamps (computed by the process that issued the request), ane if the timestamps are equal, the process with the smaller ID will win. The timestamps (clocks) on the processes are managed as scalar clocks (Lamport).

2 Hypotheses

The connections between nodes (vertices in the graph, representing processes) are bi-directional and reliable.

3 Algorithm

The algorithm is a transcription of [1].

```
============================================================
MUTUAL EXCLUSION ALGO on a CLIQUE
============================================================

// COMMUNICATION INTERFACES
ClientItf RequestQ; //interface receiving client requests (outside world)
CommItfSet S; // connection links to the neighbour nodes

// TYPES of MESSAGES
//Client request
Message_t LOCK;
Message_t UNLOCK;

//Request message to be broadcast inside the system
struct Request{
   int src // the ID of the asking process
   int d; // date of the request
```

*A clique is a complete graph in which every two distinct vertices are adjacent

```
};

//Response when a process agrees to let another process in the CS
struct OK{
   int d; // date of the response, used to synchronize clocks
};

// STATE VARIABLES
int N;               //number of processes in the system
bool Asking = false; //no process asks for the CS iat the boot of the system
int LC = 0;          //the Logical Clock, initialized at 0
ind LD = -1;         //the date of the Last Demand for the CS of this process
int nb_perm = 0;     //number of received permissions
//Delayed[i] = true iff both this process and process i
//have requested the CS and this process is prioritary
//meaning that it delays the access of process i to the CS
bool Delayed[N]= {false, ..., false};

// COMPARISON BETWEEN REQUESTS
// If id1 and id2 are two identifiers of two processes
// and d1 and d2 are two dates
// we will note (id1, d1) << (id2,d2) iff d1 < d2 || (d1==d2  && id1 < id2)
// i.e process id1 is prioritary


//-------------------------------------------------------------------
// Rule 1 : the client issues a LOCK request
//          we decide that this is blocking until the callback
//          RequestQ.REPLY is called
//-------------------------------------------------------------------
RequestQ ? request==<LOCK> ==>
   LD = LC;
   for each n in S, n!<Request(myID, LD)>
   Asking = true;
   nb_perm = 0;

//-------------------------------------------------------------------
// Rule 2 : receive a Request message while not Asking
//-------------------------------------------------------------------
exists s in S s? message == <Request(id,ld)> && !Asking ==>
   LC = max(LC, ld) + 1; //synchronize clocks
   s!<OK(LC)>;           // we say "you can go"

//-------------------------------------------------------------------
// Rule 3 : receive a Request message while Asking and having priority
//-------------------------------------------------------------------
exists s in S s? message == <Request(id,ld)> && Asking && (myId,LD) << (id,ld) ==>
   LC = max(LC, ld) + 1; //synchronize clocks
   Delayed[id] = true;   //do not send permission and remember it

//-------------------------------------------------------------------
// Rule 4 : receive a Request message while Asking and NOT having priority
//-------------------------------------------------------------------
exists s in S s? message == <Request(id,ld)> && Asking && (myId,LD) >> (id,ld) ==>
```

```
   LC = max(LC, ld) + 1; //synchronize clocks
   s!<OK(LC)>;           //send permission (it will delay me and send me a permission later)

//------------------------------------------------------------------
// Rule 5 : receive a OK message
//------------------------------------------------------------------
exists s in S s? message == <OK(d)> ==>
   LC = max(LC, ld) + 1; //synchronize clocks
   nb_perm++;
   if nb_perm == N-1    //I have all the permissions
      RequestQ.REPLY    //unblock the client i.e it can access the CS

//------------------------------------------------------------------
// Rule 6 : client issues UNLOCK request
//------------------------------------------------------------------
RequestQ ? request==<UNLOCK> ==>
   for each n in Delayed
       if Delayed[n]
          n!<OK(LC)>
          Dealyed[n]=false
   Asking = false;
   nb_perm = 0;
```

Références

[1] Ricart, Glenn and Ashok K. Agrawala. “An Optimal Algorithm for Mutual Exclusion in Computer Networks.” Commun. ACM 24 (1981) : 9-17.